

Experiment-5

Aim :

To create SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive CTE to display reporting chains.

Description :

A SQL view is a virtual table based on the result of a SELECT query. Views are useful for presenting selected data, simplifying complex queries, and providing controlled access to database information.

In this experiment, two views are created. The first view, `department_salary_summary`, groups employees by department and displays the number of employees, total salary, and average salary. The second view, `employee_hierarchy`, displays employee details together with their manager information. The updatability of a simple employee view is then tested using an UPDATE statement.

A recursive Common Table Expression (CTE) is also implemented to display the complete reporting chain of employees. The recursive CTE starts from top-level employees who do not report to another employee and repeatedly joins employees with their managers.

The following SQL uses MySQL 8.0 or later syntax because recursive CTEs require support for WITH RECURSIVE.

Source Code :

```
-- Create database objects for the experiment
```

```
CREATE TABLE department (  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE employee (  
    employee_id INT PRIMARY KEY,  
    employee_name VARCHAR(100) NOT NULL,  
    department_id INT,  
    manager_id INT NULL,  
    salary DECIMAL(10,2),  
    FOREIGN KEY (department_id) REFERENCES department(department_id),  
    FOREIGN KEY (manager_id) REFERENCES employee(employee_id)  
);
```

```
-- Insert sample departments
```

```
INSERT INTO department VALUES  
(1, 'IT'),  
(2, 'HR'),
```

(3, 'Finance');

-- Insert sample employees

```
INSERT INTO employee VALUES
(101, 'Amit', 1, NULL, 90000),
(102, 'Neha', 1, 101, 65000),
(103, 'Rahul', 1, 101, 60000),
(104, 'Priya', 2, NULL, 80000),
(105, 'Karan', 2, 104, 55000),
(106, 'Riya', 3, NULL, 85000),
(107, 'Arjun', 3, 106, 50000);
```

-- 1. View for department salary summary

```
CREATE VIEW department_salary_summary AS
SELECT
    d.department_id,
    d.department_name,
    COUNT(e.employee_id) AS employee_count,
    SUM(e.salary) AS total_salary,
    AVG(e.salary) AS average_salary
FROM department d
LEFT JOIN employee e
    ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name;

SELECT * FROM department_salary_summary;
```

-- 2. View for employee hierarchy

```
CREATE VIEW employee_hierarchy AS
SELECT
    e.employee_id,
    e.employee_name,
    e.department_id,
    e.manager_id,
    m.employee_name AS manager_name,
    e.salary
FROM employee e
LEFT JOIN employee m
    ON e.manager_id = m.employee_id;

SELECT * FROM employee_hierarchy;
```

-- 3. Test view updatability using a simple single-table view

```
CREATE VIEW employee_basic AS
SELECT employee_id, employee_name, salary
```

```
FROM employee;
```

```
-- Update through the view
```

```
UPDATE employee_basic  
SET salary = salary + 5000  
WHERE employee_id = 102;
```

```
SELECT * FROM employee_basic  
WHERE employee_id = 102;
```

```
-- 4. Recursive CTE to display reporting chains
```

```
WITH RECURSIVE reporting_chain AS (  
  -- Anchor member: top-level employees  
  SELECT  
    employee_id,  
    employee_name,  
    manager_id,  
    0 AS level,  
    CAST(employee_name AS CHAR(1000)) AS reporting_path  
  FROM employee  
  WHERE manager_id IS NULL  
  
  UNION ALL  
  
  -- Recursive member: find employees reporting to each employee  
  SELECT  
    e.employee_id,  
    e.employee_name,  
    e.manager_id,  
    rc.level + 1,  
    CONCAT(rc.reporting_path, ' -> ', e.employee_name)  
  FROM employee e  
  INNER JOIN reporting_chain rc  
    ON e.manager_id = rc.employee_id  
)  
SELECT  
  employee_id,  
  employee_name,  
  manager_id,  
  level,  
  reporting_path  
FROM reporting_chain  
ORDER BY reporting_path;
```

Output :

Run the SQL statements and capture screenshots showing the department salary summary, employee hierarchy, successful update through the simple view, and recursive reporting chain.

Screenshot 1: Department Salary Summary

The screenshot shows the byteXL SQL editor interface. The left pane displays a SQL query in a file named 'queries.sql'. The query creates a view 'department_salary_summary' and then selects data from it. The right pane shows the output of the query, which is a table with 5 columns: department_id, department_name, employee_count, total_salary, and average_salary. The output shows data for three departments: IT, HR, and Finance.

```
32 (105, 'Karan', 2, 104, 55000),
33 (106, 'Riya', 3, NULL, 85000),
34 (107, 'Arjun', 3, 106, 50000);
35
36 -- 1. View for department salary summary
37
38 CREATE VIEW department_salary_summary AS
39 SELECT
40     d.department_id,
41     d.department_name,
42     COUNT(e.employee_id) AS employee_count,
43     SUM(e.salary) AS total_salary,
44     AVG(e.salary) AS average_salary
45 FROM department d
46 LEFT JOIN employee e
47     ON d.department_id = e.department_id
48 GROUP BY d.department_id, d.department_name;
49
50 SELECT * FROM department_salary_summary;
51
52
```

Output:

department_id	department_name	employee_count	total_salary	average_salary
1	IT	3	215000.00	71666.6666
2	HR	2	135000.00	67500.0000
3	Finance	2	135000.00	67500.0000

Screenshot 2: Employee Hierarchy

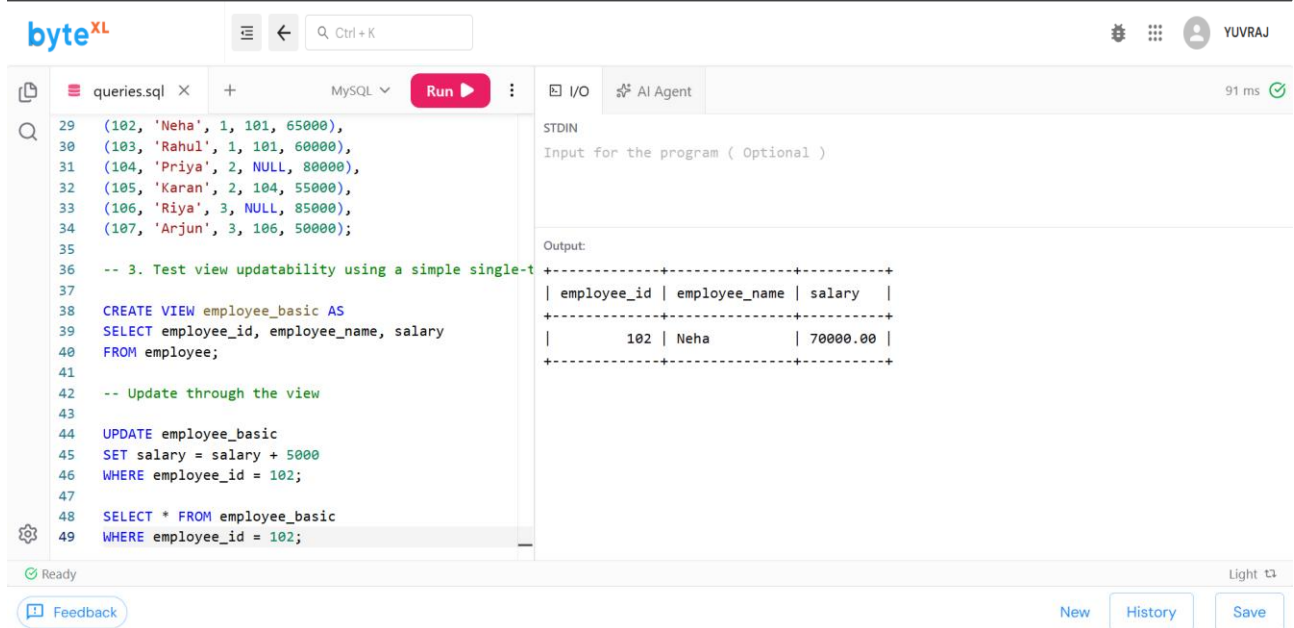
The screenshot shows the byteXL SQL editor interface. The left pane displays a SQL query in a file named 'queries.sql'. The query inserts data into an 'employee' table and then creates a view 'employee_hierarchy' that shows the hierarchy of employees based on their manager. The right pane shows the output of the query, which is a table with 6 columns: employee_id, employee_name, department_id, manager_id, manager_name, and salary. The output shows a hierarchy where employees are listed with their respective managers.

```
27 INSERT INTO employee VALUES
28 (101, 'Amit', 1, NULL, 90000),
29 (102, 'Neha', 1, 101, 65000),
30 (103, 'Rahul', 1, 101, 60000),
31 (104, 'Priya', 2, NULL, 80000),
32 (105, 'Karan', 2, 104, 55000),
33 (106, 'Riya', 3, NULL, 85000),
34 (107, 'Arjun', 3, 106, 50000);
35
36 CREATE VIEW employee_hierarchy AS
37 SELECT
38     e.employee_id,
39     e.employee_name,
40     e.department_id,
41     e.manager_id,
42     m.employee_name AS manager_name,
43     e.salary
44 FROM employee e
45 LEFT JOIN employee m
46     ON e.manager_id = m.employee_id;
47
48 SELECT * FROM employee_hierarchy;
```

Output:

employee_id	employee_name	department_id	manager_id	manager_name	salary
101	Amit	1	NULL	NULL	90000.00
102	Neha	1	101	Amit	65000.00
103	Rahul	1	101	Amit	60000.00
104	Priya	2	NULL	NULL	80000.00
105	Karan	2	104	Priya	55000.00
106	Riya	3	NULL	NULL	85000.00
107	Arjun	3	106	Riya	50000.00

Screenshot 3: View Updatability Test



The screenshot shows the byteXL IDE interface. The left pane contains a MySQL query file named 'queries.sql'. The query is as follows:

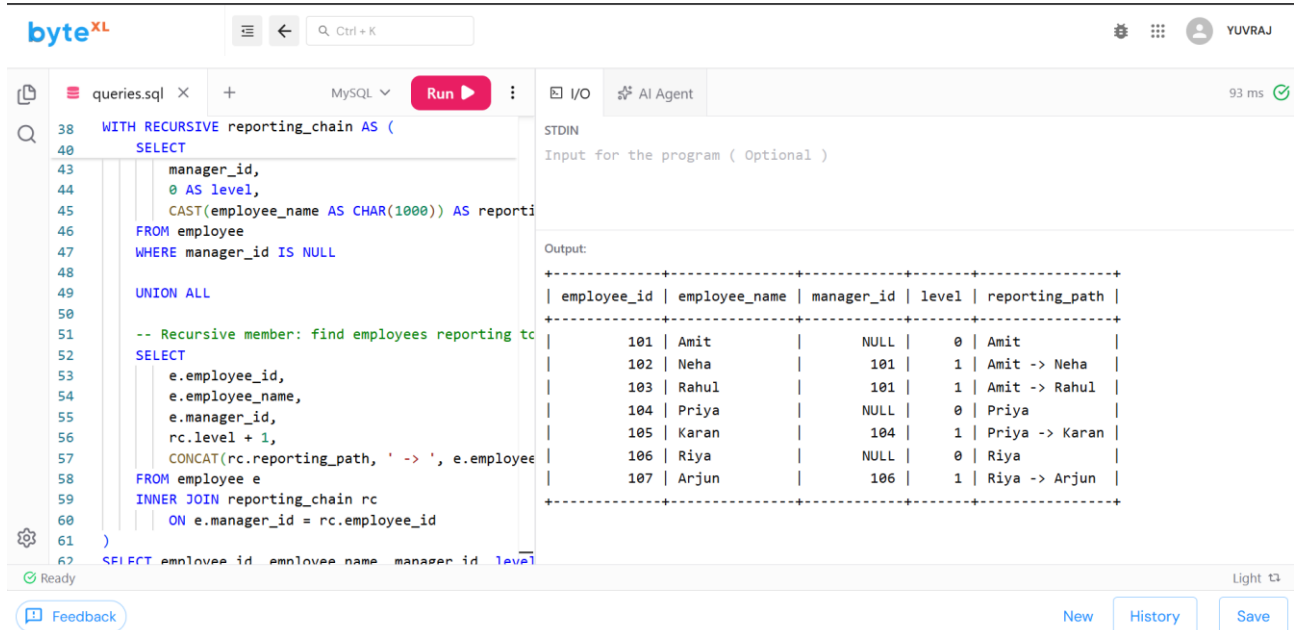
```
29 (102, 'Neha', 1, 101, 65000),
30 (103, 'Rahul', 1, 101, 60000),
31 (104, 'Priya', 2, NULL, 80000),
32 (105, 'Karan', 2, 104, 55000),
33 (106, 'Riya', 3, NULL, 85000),
34 (107, 'Arjun', 3, 106, 50000);
35
36 -- 3. Test view updatability using a simple single-t
37
38 CREATE VIEW employee_basic AS
39 SELECT employee_id, employee_name, salary
40 FROM employee;
41
42 -- Update through the view
43
44 UPDATE employee_basic
45 SET salary = salary + 5000
46 WHERE employee_id = 102;
47
48 SELECT * FROM employee_basic
49 WHERE employee_id = 102;
```

The right pane shows the output of the query. It displays the updated salary for employee 102 (Neha) as 70000.00.

employee_id	employee_name	salary
102	Neha	70000.00

The interface also shows a 'Run' button, a search bar, and a status bar indicating 'Ready'.

Screenshot 4: Recursive CTE Reporting Chain



The screenshot shows the byteXL IDE interface. The left pane contains a MySQL query file named 'queries.sql'. The query is as follows:

```
38 WITH RECURSIVE reporting_chain AS (
40     SELECT
43         manager_id,
44         0 AS level,
45         CAST(employee_name AS CHAR(100)) AS reporti
46     FROM employee
47     WHERE manager_id IS NULL
48
49     UNION ALL
50
51     -- Recursive member: find employees reporting to
52     SELECT
53         e.employee_id,
54         e.employee_name,
55         e.manager_id,
56         rc.level + 1,
57         CONCAT(rc.reporting_path, ' -> ', e.employee
58     FROM employee e
59     INNER JOIN reporting_chain rc
60         ON e.manager_id = rc.employee_id
61 )
62 SELECT employee_id, employee_name, manager_id, level,
```

The right pane shows the output of the query. It displays the reporting chain for each employee, showing the employee_id, employee_name, manager_id, level, and reporting_path.

employee_id	employee_name	manager_id	level	reporting_path
101	Amit	NULL	0	Amit
102	Neha	101	1	Amit -> Neha
103	Rahul	101	1	Amit -> Rahul
104	Priya	NULL	0	Priya
105	Karan	104	1	Priya -> Karan
106	Riya	NULL	0	Riya
107	Arjun	106	1	Riya -> Arjun

The interface also shows a 'Run' button, a search bar, and a status bar indicating 'Ready'.